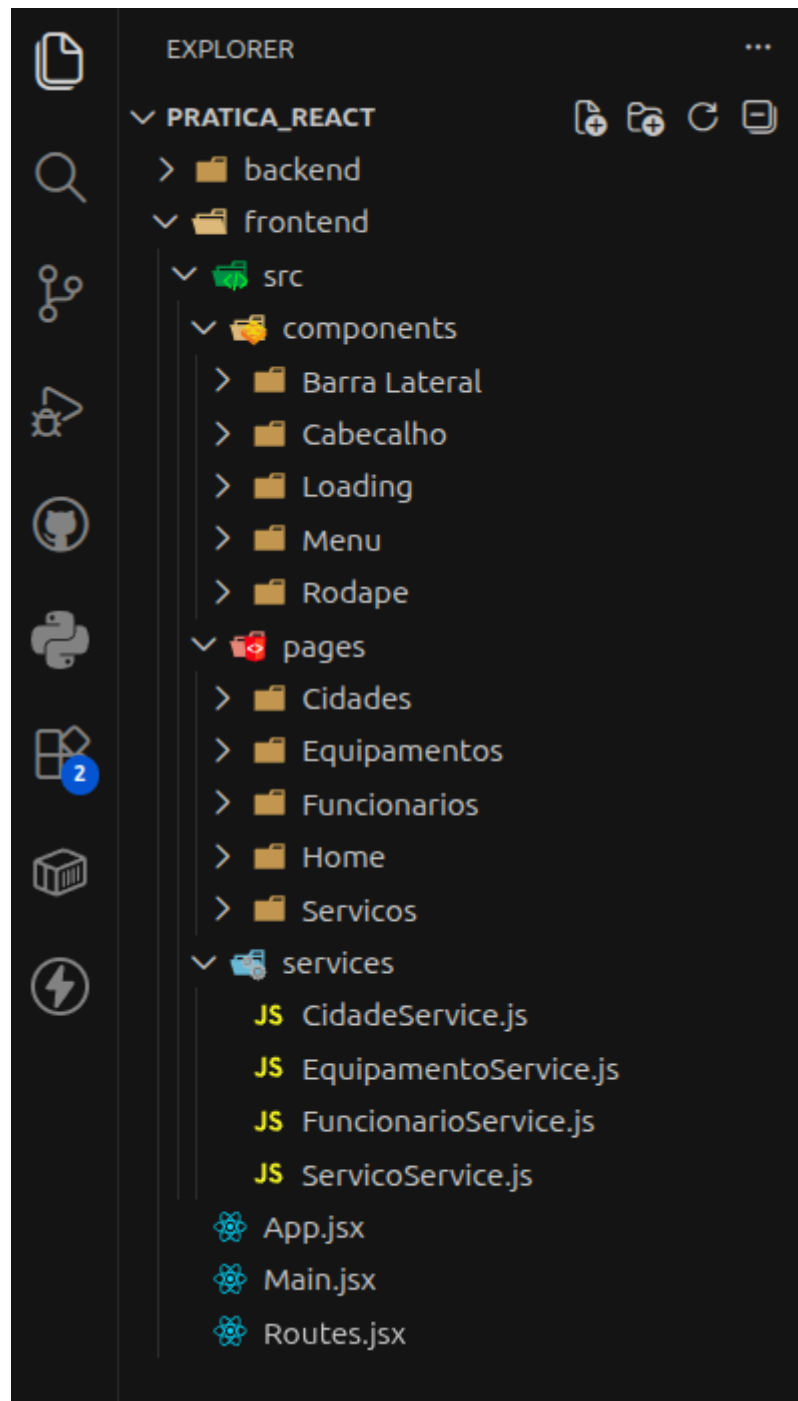
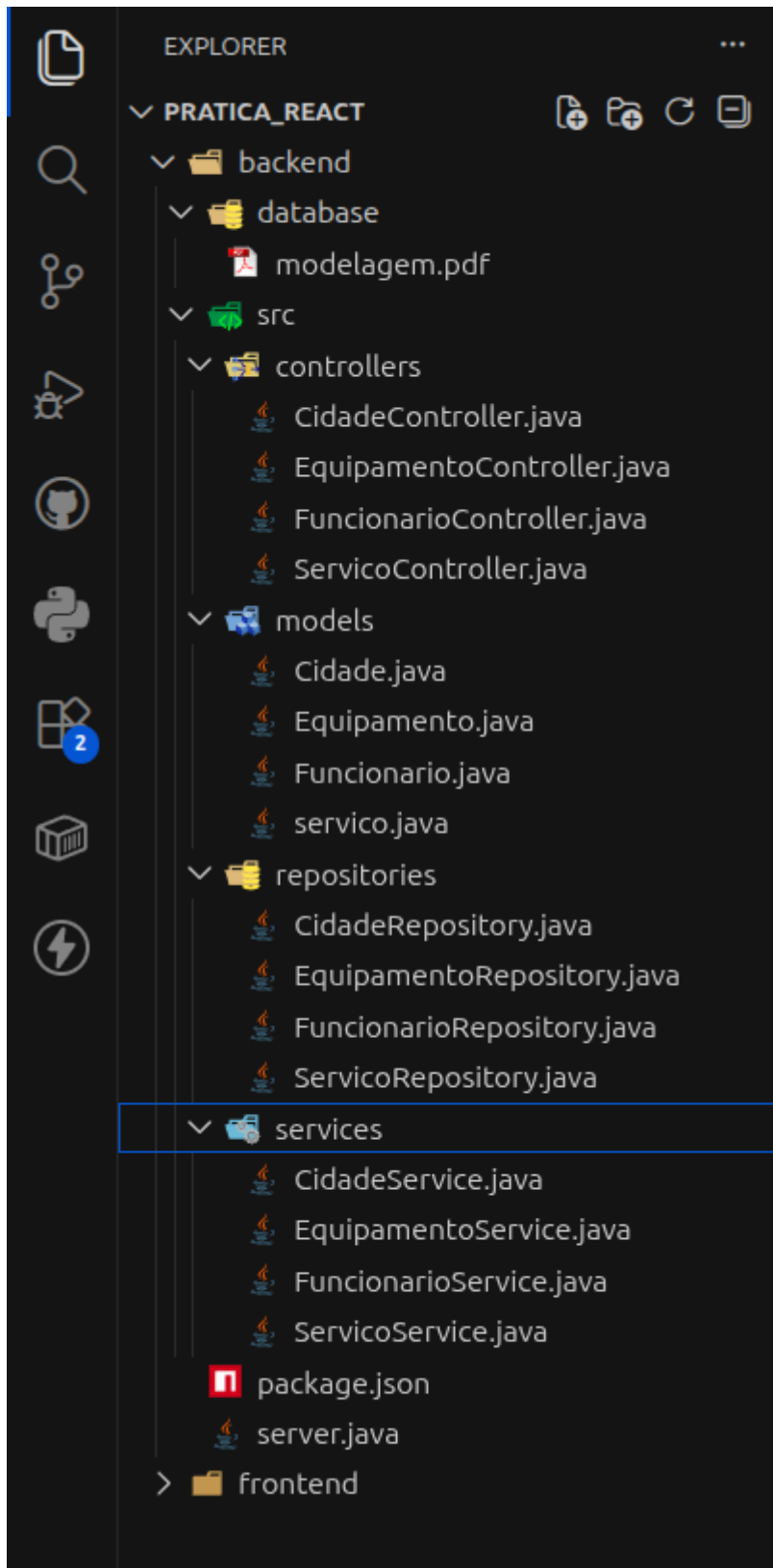


Sistema Mineradora

- **PARTE 1 - Estrutura do Front-end:**



- **PARTE 2 - Estrutura do Back-end:**



Responsabilidades de cada pasta (Frontend)	
PASTA	Responsabilidade
Frontend	Contém toda a estrutura do frontend: Páginas, componentes, serviços, rotas e etc.
src	É responsável por armazenar de forma organizada o código fonte do frontend, separando-o dos arquivos de configuração.
components	É responsável por armazenar todos os componentes utilizados nas páginas dos projetos.
pages	É responsável por armazenar os arquivos de páginas do projeto.
services	É responsável por armazenar os arquivos que se comunicam diretamente com o backend por meio de requisições.

Responsabilidades de cada pasta (Backend)	
PASTA	Responsabilidade
src	Contém toda a estrutura do backend: Entidades(models), controllers, serviços, repositories, banco de dados e etc.
models	É responsável por armazenar as entidades do sistema, objetos que correspondem a tabelas no banco de dados.
controllers	É responsável por armazenar os arquivos que recebem requisições, se comunicando diretamente com o frontend.
services	É responsável por armazenar os arquivos que definem as regras de negócio do sistema, sendo a ponte entre os controllers e repositories
repositories	É responsável por armazenar os arquivos que conversam diretamente com o banco de dados realizando alterações e consultas.
database	Armazena o banco de dados do sistema, sua lógica e configurações.

- **PARTE 3 - Estrutura do banco de dados:**

```
1  BANCO DE DADOS
2
3  Cidade
4      - id
5      - nome
6      - estado
7      - população
8      - endereço
9
10  Funcionário
11      - id
12      - nome
13      - cpf
14      - cargo
15      - telefone
16      - salario
17      - cidade_id
18
19  Equipamento
20      - id
21      - nome
22      - tipo
23      - modelo
24      - fabricante
25      - funcionario_id
26      - data_compra
27      - status
28
29  Serviço
30      - id
31      - nome
32      - descricao
33      - data_inicio
34      - data_fim
35      - status
36
37  Servico_Equipamento
38      - id
39      - servico_id
40      - equipamento_id
41
42  Relacionamentos
43      - Cidade (1) ----- (n) Funcionario
44      | Uma cidade pode ter vários funcionários
45      - Funcionario (1) ----- (n) Equipamento
46      | Um funcionario pode utilizar vários equipamentos
47      - Servico (n) ----- (n) Equipamento
48      | Um serviço pode utilizar vários equipamentos e vice versa
```

● **PARTE 4 - Endpoints da API:**

Equipamentos

- Cadastrar Equipamento
- Listar Equipamentos
- Buscar Equipamento por ID
- Editar Equipamento
- Excluir Equipamento

Funcionários

- Cadastrar Funcionário
- Listar Funcionários
- Buscar Funcionário por ID
- Editar Funcionário
- Excluir Funcionário

Cidades

- Cadastrar Cidade
- Listar Cidades
- Buscar Cidade por ID
- Editar Cidade
- Excluir Cidade

Serviços

- Cadastrar Serviço
- Listar Serviços
- Buscar Serviço por ID
- Editar Serviço
- Excluir Serviço

● **PARTE 5 - Planejamento do projeto:**

1 - O módulo de Cidades é o mais simples de desenvolver, pois possui poucos atributos e relacionamentos simples.

2 - O módulo de Serviços é o mais complexo, pois pode envolver relacionamentos com equipamentos, além de informações como descrição, período de execução e status.

3 - Ordem:

- 1 - Cidade
- 2 - Funcionário
- 3 - Equipamento
- 4 - Serviço

Justificativa: A implementação deve começar pelo módulo de Cidades que serve de base para o cadastro de funcionários e equipamentos. Em seguida, o módulo de Funcionários e o módulo de Equipamentos, que dependem das cidades já cadastradas, depois, desenvolve o módulo de Serviços, pois ele possui mais relacionamentos e depende das informações dos equipamentos para funcionar corretamente.